API Security Risk Analysis Report

Cybersecurity Internship — Task 3

Read-Only Audit of Public REST APIs

Report Classification	Confidential — Educational Use Only
APIs Under Test	JSONPlaceholder & ReqRes
Audit Type	Read-Only Passive Security Inspection
Total Findings	6 (1 Critical · 2 High · 2 Medium · 1 Low)
Overall Risk Rating	HIGH RISK
Analyst	Utpal
Report Date	26 April 2026

Table of Contents

1 — Executive Summary
2 — APIs Under Test
3 — Risk Findings Summary
4 — Detailed Findings
4.1 — F-01 · Unauthenticated Access to User Data
4.2 — F-02 · Broken Object Level Authorization (BOLA)
4.3 — F-03 · Excessive Data Exposure
4.4 — F-04 · Missing Rate Limiting
4.5 — F-05 · Absent HTTP Security Headers
4.6 — F-06 · Auth Token Not Enforced (ReqRes)
5 — Remediation Roadmap
6 — Conclusion
Appendix A — Severity Classification Guide
Appendix B — OWASP API Security Top 10 Mapping

1. Executive Summary

This report presents the findings of a read-only API Security Risk Analysis performed on two publicly available demo APIs: JSONPlaceholder (jsonplaceholder.typicode.com) and ReqRes (reqres.in). The assessment was conducted as part of Cybersecurity Internship Task 3 and followed the OWASP API Security Top 10 (2023 edition) as its primary reference framework.

Six security vulnerabilities were identified across the two target APIs. One finding is rated Critical, two are rated High, two are rated Medium, and one is rated Low. The Critical finding — complete unauthenticated access to a full dataset of user records — represents an immediate, real-world data breach of risk in any production equivalent of this architecture. Left unaddressed, this single finding alone would expose all user PII (Personally Identifiable Information) to any party capable of sending an HTTP GET request.

The overall security posture of the tested APIs is assessed as HIGH RISK. Organisations operating production APIs with similar design patterns should treat this report's remediation roadmap as a priority engineering initiative. All six findings have well-understood, low-to-medium effort fixes that follow industry-standard practices.

CRITICAL	HIGH	MEDIUM	LOW
1 Finding	2 Findings	2 Findings	1 Finding

NOTE Scope limitation: This assessment was conducted on public demo APIs intentionally designed for testing. All tests were read-only. The findings document vulnerability patterns applicable to real-world production APIs — no systems were harmed or exploited.

2. APIs Under Test

The following two APIs were selected for this assessment. Both are publicly available, explicitly designed for developer testing and learning, and carry no expectation of security hardening — making them excellent case studies for identifying and communicating real-world API risk patterns.

2.1 JSONPlaceholder

Attribute	Value
Full Name	JSONPlaceholder — Free Fake REST API for Testing
Base URL	https://jsonplaceholder.typicode.com
Operator	Typicode (open-source, publicly hosted)
Purpose	Provides fake REST API responses for prototyping and testing front-end apps
Auth Scheme	None — all endpoints are publicly accessible
Data Format	JSON (application/json)
Resources	/users, /posts, /comments, /albums, /photos, /todos
Tests Run	F-01 (Unauth Access), F-02 (BOLA), F-03 (Data Exposure), F-04 (Rate Limiting), F-05 (Headers)

JSONPlaceholder serves ten fake user records with rich detail: full names, email addresses, physical addresses (including geo-coordinates), phone numbers, website URLs, and company metadata. This breadth of data makes it an ideal target for testing both unauthenticated access (F-01) and excessive data exposure (F-03).

2.2 ReqRes

Attribute	Value
Full Name	ReqRes — Hosted REST API for Front-End Testing
Base URL	https://reqres.in
Operator	Ben Howdle (open-source, publicly hosted)
Purpose	Simulates a real API with authentication flows, paginated users, and CRUD operations
Auth Scheme	Token-based — POST /api/login returns a Bearer token
Data Format	JSON (application/json)
Resources	/api/users, /api/login, /api/register
Tests Run	F-04 (Rate Limiting), F-05 (Headers), F-06 (Token Enforcement)

ReqRes is notable because it provides an authentication mechanism — a login endpoint that issues tokens — while failing to enforce those tokens on subsequent requests. This inconsistency is the basis for Finding F-06 and illustrates a common real-world pattern where authentication is implemented but not uniformly applied.

3. Risk Findings Summary

The table below provides a consolidated one-page overview of all six findings. Each finding is assigned to a unique identifier, severity rating, OWASP reference, affected API and endpoint, and a brief description of the risk. Detailed analysis of each finding follows in Section 4.

ID	Title	Severity	OWASP Ref	Brief Description
F-01	**Unauthenticated Access**	CRITICAL	API2:2023	All user PII accessible with zero authentication
F-02	**Broken Object Level Auth**	HIGH	API1:2023	Any user record accessible by changing numeric ID
F-03	**Excessive Data Exposure**	HIGH	API3:2023	Response includes geo, company data not needed by caller
F-04	**Missing Rate Limiting**	MEDIUM	API4:2023	No throttling on any endpoint; scraping and DoS risk
F-05	**Absent Security Headers**	MEDIUM	API8:2023	5 standard security response headers missing
F-06	**Token Not Enforced**	LOW	API2:2023	Auth token issued but not checked on user list endpoint

4. Detailed Findings

Each finding below follows the DEBA structure: Description (what the vulnerability is), Evidence (proof from actual HTTP requests and responses), Business Impact (why it matters to the organisation), and Remediation (specific steps to fix it). Findings are ordered by severity — most critical first.

4.1 F-01 — Unauthenticated Access to User Data

F-01	Unauthenticated Access to User Data		CRITICAL
OWASP Ref	OWASP API2:2023 — Broken Authentication	**API / Endpoint**	JSONPlaceholder — GET /users

Description

The /users endpoint returns a complete dataset of all user records — including full names, email addresses, physical addresses, geographic coordinates, phone numbers, and company information — without requiring any form of authentication. A GET request sent with no Authorization header, no API key, and no session token receives a 200 OK response with all 10 user records. This is the most severe finding in this assessment.

Evidence

```
Request:  GET https://jsonplaceholder.typicode.com/users
Headers:  (none — no Authorization header)
Response: HTTP 200 OK

[ { "id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz",
    "address": { "street": "Kulas Light", "city": "Gwenborough",
      "geo": { "lat": "-37.3159", "lng": "81.1496" } },
    "phone": "1-770-736-8031 x56442", "website": "hildegard.org",
    "company": { "name": "Romaguera-Crowe" } },
  ... 9 more user records
]
```

Business Impact

- Complete PII harvest in a single unauthenticated request: an attacker obtains names, emails, addresses, and phone numbers for all users with zero prior access.
- Regulatory exposure: In India, the DPDP Act 2023 requires organisations to protect personal data. A breach via this vector would trigger mandatory notification of obligations and potential penalties.
- In a production equivalent (e-commerce, SaaS, healthcare), this finding alone would constitute a reportable data breach affecting every registered user.
- Automated scraping: Bots can harvest the full dataset in milliseconds, building spam or phishing lists from real user data.

Remediation Steps

1. Require a valid Bearer token or session cookie on all endpoints that return user data. The server must return HTTP 401 Unauthorized when no valid credential is presented.
2. Implement authentication of middleware that runs before any route handler. In FastAPI: use Depends(verify_token) on the route definition.
3. Conduct a full endpoint audit to identify any other routes that return sensitive data without an authentication check.
4. Add automated regression tests that assert a 401 response when endpoints are called without credentials, preventing accidental removal of auth checks in future deployments.

4.2 F-02 — Broken Object Level Authorization (BOLA)

F-02	Broken Object Level Authorization (BOLA)		HIGH
OWASP Ref	OWASP API1:2023 — Broken Object Level Authorization	**API / Endpoint**	JSONPlaceholder — GET /users/{id}

Description

The /users/{id} endpoint accepts any integer as the user ID and returns the corresponding user record without verifying whether the requesting client is authorised to access that specific record. By incrementing the ID in the URL path from /users/1 to /users/2, /users/3, and beyond, an authenticated attacker can enumerate and retrieve the records of all other users. BOLA is the number-one vulnerability in the OWASP API Security Top 10.

Evidence

```
GET /users/1  →  200 OK  { id:1, name:'Leanne Graham', email:'Sincere@april.biz' }
GET /users/2  →  200 OK  { id:2, name:'Ervin Howell',
email:'Shanna@melissa.tv'  }
GET /users/3  →  200 OK  { id:3, name:'Clementine B.',
email:'Nathan@yesenia.net' }

All requests: no Authorization header, no session — 200 OK each time.
Sequential integer IDs (1, 2, 3 ...) make enumeration trivial.
```

Business Impact

- Any authenticated user can access every other user's complete profile record, bypassing the privacy boundary between accounts.
- In banking or healthcare APIs, this pattern would expose account balances, medical records, or personally sensitive data across all customers.
- The sequential numeric IDs make full enumeration scriptable in minutes — a single for loop from 1 to N retrieves the entire user database.
- Difficult to detect: BOLA attacks look identical to legitimate requests in access logs. Organisations often discover this vulnerability only after a data breach is reported.

Remediation Steps

5. Implement object-level authorisation checks in every endpoint handler. Verify that the authenticated user's token-derived ID matches the ID of the requested resource before returning data.
6. Replace sequential integer IDs with UUID v4 identifiers (e.g. 550e8400-e29b-41d4-a716-446655440000). UUIDs are 128-bit random values — guessing a valid UUID is computationally infeasible.
7. Apply a consistent authorisation middleware rather than checking permissions per-endpoint. Middleware-level enforcement ensures new endpoints are protected by default.
8. Implement anomaly detection to alert on access patterns suggesting enumeration (e.g. the same IP accessing 50+ distinct user IDs in 60 seconds).

4.3 F-03 — Excessive Data Exposure

F-03	Excessive Data Exposure		HIGH
OWASP Ref	OWASP API3:2023 — Broken Object Property Level Authorization	**API / Endpoint**	JSONPlaceholder — GET /users/{id}

Description

The single-user endpoint returns a significantly wider field set than any typical API consumer would require. A mobile app displaying a user profile needs the user's name and email. The response additionally includes geographic coordinates (latitude and longitude), full physical address, personal website URL, and company internal metadata fields (bs and catchPhrase). Returning unnecessary fields violates the principle of data minimisation and expands the blast radius of any other vulnerability that grants access to this endpoint.

Evidence

```
GET /users/1  →  200 OK
{
  "id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light", "city": "Gwenborough",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }   <-- geo-coordinates
  },
  "phone": "1-770-736-8031",
  "website": "hildegard.org",                        <-- personal website
  "company": {
    "name": "Romaguera-Crowe",
    "bs": "harness real-time e-markets",             <-- internal metadata
    "catchPhrase": "Multi-layered neural-net"        <-- internal metadata
  }
}
```

Business Impact

- Precise geographic coordinates expose users' home or work locations to anyone who can access the endpoint — a serious personal safety concern.
- Over-exposed fields increase the intelligence an attacker obtains from any other vulnerability. Combined with F-01 (unauthenticated access), all geo-coordinates for all users are publicly available.
- Internal metadata fields (bs, catchPhrase) may reveal system design or categorisation logic that aids targeted attacks.
- Privacy regulation risk: returning PII fields that are not necessary for the stated purpose of the endpoint may constitute a DPDP Act 2023 violation.

Remediation Steps

9. Define separate response DTOs (Data Transfer Objects) for each endpoint. The /users/{id} endpoint should return only the fields its documented consumers actually need.
10. Conduct a field-level audit of every API response schema: for each field, document which client use case requires it. Remove fields with no active use case.
11. Never return raw database model objects directly from API endpoints — always map to a scoped response schema.
12. Implement API response linting in CI/CD pipelines that fails the build if a response schema includes fields in a deny-list (e.g. geo, internal metadata keys).

4.4 F-04 — Missing Rate Limiting

F-04	Missing Rate Limiting on All Endpoints		MEDIUM
OWASP Ref	OWASP API4:2023 — Unrestricted Resource Consumption	**API / Endpoint**	JSONPlaceholder & ReqRes — All endpoints

Description

Five consecutive GET requests were sent to /users with 200-millisecond gaps between each. None of the five responses contained any rate-limit headers (X-RateLimit-Limit, X-RateLimit-Remaining, Retry-After, or RateLimit-Limit). No HTTP 429 Too Many Requests response was received. The absence of rate limiting means there is no server-side control preventing an attacker from sending thousands of requests per minute to scrape data, perform credential stuffing, or cause a denial of service.

Evidence

```
Requests 1-5:  GET /users  →  200 OK each time

Response headers inspected — none of these were present:
  X-RateLimit-Limit       (not found)
  X-RateLimit-Remaining   (not found)
  Retry-After             (not found)
  RateLimit-Limit         (not found)
```

```
No HTTP 429 response received across any of the 5 requests.
Note: test was limited to 5 requests at 200ms intervals — ethical scope only.
```

Business Impact

- Data scraping: without rate limiting, a single attacker script can harvest the entire dataset in seconds, trivially bypassing the intended purpose of the API.
- Credential stuffing: login endpoints with no rate limiting can be targeted with millions of username/password pairs from leaked credential databases, compromising user accounts at scale.
- Denial of Service: a flood of requests can exhaust server resources, taking the API offline for legitimate users.
- Cost amplification in cloud environments: API calls that trigger expensive backend operations (database queries, ML inference) can generate unexpected cost spikes when not rate-limited.

Remediation Steps

13. Implement rate limiting at the API gateway or reverse proxy layer (e.g. Nginx limit_req, AWS API Gateway usage plans, Kong rate-limiting plugin). This avoids per-endpoint implementation and applies globally.
14. Apply differentiated rate limits by endpoint sensitivity: authentication endpoints (100/minute), public data endpoints (300/minute), admin endpoints (10/minute).
15. Return HTTP 429 Too Many Requests with a Retry-After header when the limit is exceeded, so well-behaved clients can back off appropriately.
16. Implement IP-based and token-based rate limiting independently, so neither mechanism alone can be bypassed by rotating IPs or using a single valid token.

4.5 F-05 — Absent HTTP Security Response Headers

F-05	Absent HTTP Security Response Headers		MEDIUM
OWASP Ref	OWASP API8:2023 — Security Misconfiguration	**API / Endpoint**	JSONPlaceholder & ReqRes — All endpoints

Description

A GET request was sent to the /users endpoint and all response headers were inspected. Five standard security headers that are widely considered baseline hygiene for any web-facing API were absent from the response. These headers instruct browsers and HTTP clients how to handle the response safely and protect against a range of well-known browser-based attack vectors including MIME sniffing, clickjacking, and cross-site scripting.

Evidence

```
GET /users  →  200 OK
```

```
Security header audit results:
  X-Content-Type-Options       MISSING  (expected: nosniff)
  X-Frame-Options              MISSING  (expected: DENY)
  Strict-Transport-Security    MISSING  (expected: max-age=31536000)
  Content-Security-Policy      MISSING  (expected: default-src 'self')
  Cache-Control                MISSING  (expected: no-store, no-cache)

Headers present in response: Content-Type, Date, Transfer-Encoding
```

Business Impact

- Without X-Content-Type-Options, attackers can exploit MIME-type sniffing to execute injected content as a different type than intended.
- Without X-Frame-Options, the API's responses can be embedded in malicious iframes, enabling clickjacking attacks that trick users into performing unintended actions.
- Without Strict-Transport-Security, browsers may downgrade connections from HTTPS to HTTP, allowing man-in-the-middle attackers to intercept API traffic and session tokens.
- Without Cache-Control: no-store, browsers and proxies may cache sensitive API responses containing user PII, exposing that data to subsequent users on shared devices.
- Compliance impact: many security frameworks (ISO 27001, SOC 2, PCI DSS) explicitly require the presence of HTTP security headers.

Remediation Steps

17. Add all five missing headers at the web server or API gateway layer so they apply globally to all responses. This is a configuration change requiring no code modification: one Nginx or Apache config block protects the entire API surface.
18. Validate header presence post-deployment using securityheaders.com or equivalent tooling. Integrate header validation into the CI/CD pipeline as a deployment gate.
19. For APIs serving JSON only, a restrictive Content-Security-Policy: default-src 'none' is a safe and complete baseline.
20. Add Cache-Control: no-store to all endpoints returning user data or authentication tokens to prevent browser caching of sensitive responses.

4.6 F-06 — Auth Token Not Enforced on User List (ReqRes)

F-06	**Auth Token Not Enforced on User List Endpoint**		**LOW**
OWASP Ref	OWASP API2:2023 — Broken Authentication	**API / Endpoint**	ReqRes — GET /api/users

Description

The ReqRes API provides an authentication mechanism: a POST /api/login endpoint that accepts credentials and returns a Bearer token. However, the GET /api/users endpoint — which returns a paginated list of user records — accepts requests without the token in the Authorization header and returns a 200 OK response with user data. The authentication step is

cosmetic: the token is issued but never enforced on downstream endpoints, rendering the entire authentication mechanism non-functional for data protection.

Evidence

```
Step 1 — Obtain token:
  POST https://reqres.in/api/login
  Body: { "email": "eve.holt@reqres.in", "password": "cityslicka" }
  Response: { "token": "QpwL5tpe83ilfN2" }  →  200 OK

Step 2 — Access user data WITHOUT the token:
  GET https://reqres.in/api/users
  Authorization: (none)
  Response: { "page":1, "data":[{...users...}] }  →  200 OK

Finding: token from Step 1 is never required in Step 2.
```

Business Impact

- The authentication mechanism provides a false sense of security: developers integrating with this API may believe data is protected by the issued token, when in fact it is not.
- Unauthenticated users or external attackers can access the user list without possessing a valid token.
- In a real application this pattern means that token revocation (e.g. on logout or account suspension) would have no effect — a revoked token holder and an unauthenticated attacker would receive identical access.
- Inconsistent auth enforcement is a maintenance risk: future developers may assume the pattern is deliberate and replicate it across new endpoints.

Remediation Steps

21. Apply authentication middleware consistently to all routes that return user data. There should be no exceptions: if a route returns data, it must require a valid token.
22. Perform a full route audit: list every endpoint and annotate whether it currently checks for a valid token. Any gap is a potential F-06 equivalent.
23. Write automated integration tests that assert a 401 response for every authenticated route when called without a token. These tests should run in CI to prevent regression.
24. Document the intended authentication model (which routes require auth, which are public) in the API specification, so the requirement is explicit and auditable.

5. Remediation Roadmap

The table below translates the six findings into a prioritised remediation roadmap. Findings are ordered by priority (Immediate → Short-Term → Planned). Implementation effort is estimated relative to a single developer for a typical SaaS backend.

ID	Finding	Priority	Severity	Effort	Recommended Action
F-01	Unauthenticated Access	Immediate	CRITICAL	Medium	Add auth middleware; return 401 for all unauthenticated data requests
F-02	BOLA / ID Enumeration	Immediate	HIGH	Medium	Object-level auth checks + migrate IDs to UUID v4
F-03	Excessive Data Exposure	Short-Term	HIGH	Low	Define scoped DTOs per endpoint; remove unused fields from responses
F-04	No Rate Limiting	Short-Term	MEDIUM	Low	Configure rate limiting at API gateway; return 429 on limit exceeded
F-05	Missing Sec. Headers	Short-Term	MEDIUM	Low	Add 5 security headers globally at web server config level
F-06	Token Not Enforced	Planned	LOW	Low	Apply token verification middleware uniformly to all data routes

NOTE Effort key: Low = less than 1 day (config change or single-file code addition); Medium = 1–3 days (middleware implementation + testing); High = 3+ days (architectural change with migration).

6. Conclusion

This API Security Risk Analysis identified six vulnerabilities across JSONPlaceholder and ReqRes, spanning four of the OWASP API Security Top 10 categories. The overall security posture is assessed as HIGH RISK, driven primarily by the unauthenticated access finding (F-01) that would constitute an immediate data breach in any production-equivalent application.

Three patterns emerge from the findings taken together. First, access control is entirely absent: there is no authentication requirement on data endpoints (F-01), no object-level authorisation preventing cross-user data access (F-02), and no enforcement of the token mechanism that does exist (F-06). This represents a systematic gap in the authorisation design. Second, the API returns more data than necessary (F-03), compounding the impact of the access control failures. Third, standard operational security controls — rate limiting (F-04) and security response headers (F-05) — are missing, removing additional defensive layers.

The remediation roadmap in Section 5 addresses all six findings. The two Immediate-priority items (F-01 and F-02) can realistically be implemented in under a week of engineering effort and would eliminate the Critical and one of the two High severity findings. The remaining four findings are Short-Term or Planned and represent configuration changes and incremental improvements rather than architectural rework.

KEY	Organisations deploying APIs in production should treat this report's Immediate items as blocking issues — no API handling real user data should go live without authentication middleware and object-level authorisation.

Appendix A: Severity Classification Guide

This report uses the following four-tier severity model, aligned with industry standards used by professional AppSec agencies and penetration testing firms.

Severity	Definition	Criteria	Required Action
CRITICAL	Immediate data breach possible with no authentication required	Immediate, unannounced access to all user data or system compromise	Fix before deployment; treat as P0 incident
HIGH	Significant access control failure; authenticated attacker can access others' data	BOLA, BFLA, or mass data exposure requiring a valid token	Fix within current sprint
MEDIUM	Security control missing or misconfigured; enables secondary attacks	Missing rate limiting, security headers, verbose errors	Fix within 30 days
LOW	Best practice deviation; low immediate risk but degrades security posture	Inconsistent token enforcement, informational leakage	Fix within 90 days

Appendix B: OWASP API Security Top 10 Mapping

The table below maps all ten categories of the OWASP API Security Top 10 (2023 edition) to their coverage status in this assessment.

OWASP ID	Vulnerability Name	Covered	Finding ID(s)
API1:2023	Broken Object Level Authorization	Yes	F-02
API2:2023	Broken Authentication	Yes	F-01, F-06
API3:2023	Broken Object Property Level Authorization	Yes	F-03
API4:2023	Unrestricted Resource Consumption	Yes	F-04
API5:2023	Broken Function Level Authorization	Partial	Scope review only
API6:2023	Unrestricted Access to Sensitive Business Flows	No	Out of scope
API7:2023	Server Side Request Forgery (SSRF)	No	Out of scope
API8:2023	Security Misconfiguration	Yes	F-05
API9:2023	Improper Inventory Management	Partial	Doc review only
API10:2023	Unsafe Consumption of APIs	No	Out of scope

— End of Report —

API Security Risk Analysis Report · · Cybersecurity Internship Task 3